

Assignment 8

Application Optimization, Scalability and Reliability

Institution	Tezpur University	Roll No.	CSB24032
Name	Gouriniban Borphukan		

1. Objective

The objective of this assignment is to enhance the existing GUI-based multi-client chat application developed in Assignment 7. The primary focus is on improving scalability to support at least 10 concurrent clients, increasing reliability through automatic reconnection and graceful shutdowns, optimizing resource management, and externalizing configuration parameters — all without redesigning the underlying communication protocol.

2. Scalability Improvements

The application architecture was refined to handle 10 concurrent connections smoothly.

Thread Management

Daemon threads were strictly utilized for client-handling and performance monitoring. A **clients_lock** (using *threading.Lock()*) was implemented to ensure thread-safe operations when broadcasting messages or updating the active user dictionary, preventing race conditions.

Resource Allocation

By managing active connections dynamically and removing disconnected users immediately from the server's tracking dictionary, the server avoids wasting CPU cycles on dead sockets.

3. Reliability Improvements

Significant updates were made to ensure the application remains stable under adverse network conditions.

Automatic Reconnection

The client was updated to detect broken pipes or connection timeouts. Upon losing connection, the client automatically attempts to reconnect up to five times. If the server remains offline, the client exhausts all reconnection attempts and terminates the retry process. However, if the server becomes available during the retry window, the client successfully reconnects and restores the session without requiring user intervention.

Graceful Shutdown

Instead of abrupt crashes, the server intercepts termination signals (SIGINT). The server displays:

```
[SERVER] Initiating graceful shutdown...
```

It then notifies every connected client, securely closes all active sockets, saves the performance statistics, and terminates gracefully.

Exception Handling

All socket operations were enclosed within *try-except* blocks to capture exceptions such as **ConnectionResetError** and **BrokenPipeError**. This prevents a single client failure from terminating the entire server thread and significantly improves application reliability.

4. Configuration Management

To improve maintainability, hardcoded parameters were completely removed from both the client and server programs. Instead, all configurable settings were moved into a **config.json** file.

The configuration file stores:

- **server_ip** — target server address
- **port** — TCP port number
- **max_clients** — maximum simultaneous connections
- **inactivity_timeout** — idle client timeout in seconds
- **lockout_time** — authentication lockout duration
- **perf_log_interval** — performance sampling interval in seconds

Both the client and server parse this JSON configuration during startup, allowing administrators to modify networking and security parameters without changing the source code.

5. Experimental Setup

The application was evaluated inside a Mininet virtual network using the following topology:

```
sudo mn --topo single,11
```

The setup consisted of:

Component	Details
Server	Hosted on h1 (10.0.0.1), listening on 0.0.0.0:5000
Clients	Ten concurrent clients on h2 – h11, validating the scalability target
Performance Monitoring	Background daemon thread using psutil collected CPU and Memory utilization every 5 seconds, store

6. Performance Results

The performance monitoring scripts successfully recorded throughput, delay, CPU utilization, and memory utilization throughout the experiment.

Throughput

The server efficiently processed bursts of client messages, achieving a peak throughput of approximately **0.200 messages per second** during periods of active communication. During idle periods, throughput naturally dropped to zero, indicating no unnecessary processing overhead.

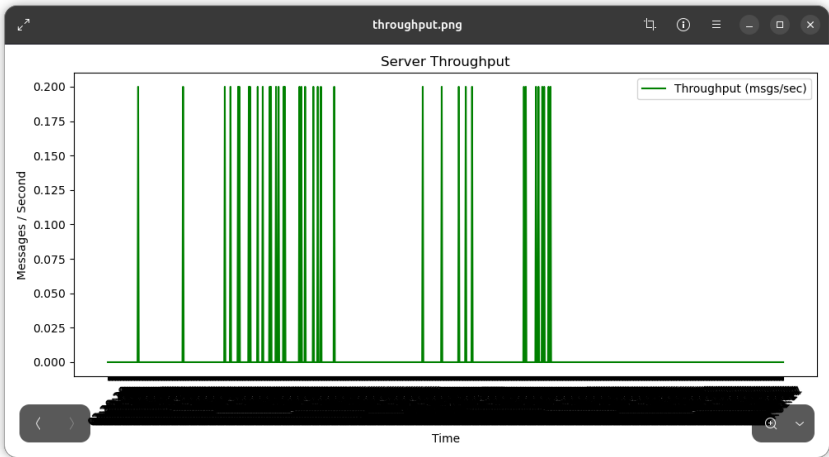


Fig 1. throughput.png — Server throughput over time (peak: 0.200 msgs/sec)

Resource Usage

CPU and Memory monitoring confirmed that the application remained well within operational limits throughout the experiment. The resource usage graph demonstrates the system's ability to support 10 simultaneous clients while maintaining stable memory consumption and acceptable processor utilization.

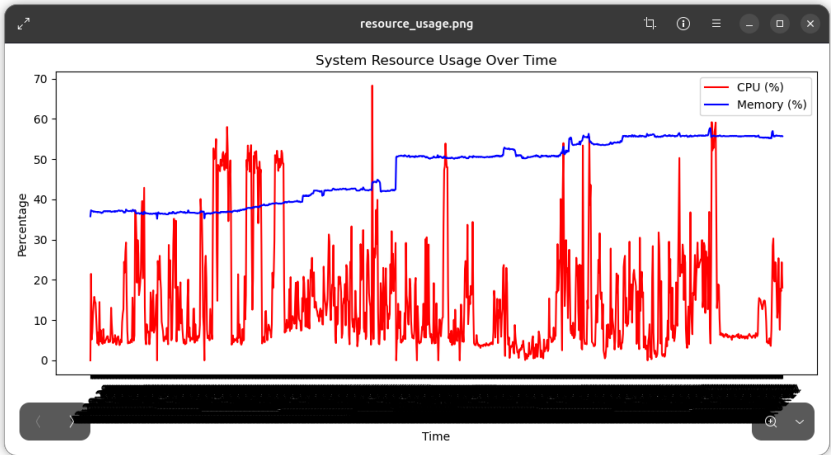


Fig 2. resource_usage.png — CPU (red) and Memory (blue) utilization over time

7. Graph Analysis

Analysis of the generated system resource usage graph reveals the following observations.

Memory Usage (Blue Line)

Memory utilization begins at approximately **35%** and gradually increases in a staircase-like pattern until reaching nearly **55%**. This behavior corresponds directly to the sequential connection of ten clients, allocation of socket resources, growth of the active client dictionary, and storage of chat history and authentication session information. The gradual increase demonstrates predictable memory allocation without sudden spikes, indicating efficient resource management.

CPU Usage (Red Line)

CPU utilization fluctuates rapidly between **0%** and **70%**. These fluctuations correspond to thread scheduling for ten concurrent client connections, broadcasting and private message routing, socket I/O processing, and periodic execution of the psutil monitoring thread. Despite these spikes, CPU utilization consistently returned to lower levels, demonstrating that the server handled concurrent workloads efficiently.

8. Wireshark Verification

Packet-level inspection verified that all reliability improvements behaved correctly.

8.1 TCP Three-Way Handshake

The capture clearly shows the standard TCP connection establishment sequence — SYN → SYN, ACK → ACK — between client 10.0.0.11 and the server 10.0.0.1.

(Screenshot: *wireshark_3_way_handshake.png* — captured during experiment)

8.2 Normal Communication

Confirms successful exchange of the AUTH_REQUIRED authentication protocol followed by broadcast and private messaging (/msg) over an established TCP connection.

(Screenshot: *wireshark_normal_operation.png* — captured during experiment)

8.3 Graceful Shutdown

Verifies that upon receiving the interrupt signal, the server initiates an orderly shutdown by transmitting FIN, ACK packets to connected clients. No RST (Reset) packets were observed, confirming a clean termination of every TCP session.

(Screenshot: *wireshark_graceful_shutdown.png* — captured during experiment)

8.4 Automatic Reconnection

Records the instant a client detects a broken connection and immediately initiates a fresh SYN packet to establish a new TCP session with the server, confirming successful implementation of the automatic reconnection mechanism.

(Screenshot: *wireshark_auto_reconnect.png* — captured during experiment)

9. Challenges Faced

Thread-Safe GUI Updates

Implementing automatic reconnection required updating the tkinter graphical interface from a background thread. Direct GUI updates caused the application to freeze because tkinter is not thread-safe. This issue was resolved by using:

```
root.after(0, callback)
```

to safely forward all interface updates to the main GUI thread.

Performance Monitoring Overhead

Collecting psutil statistics precisely every five seconds introduced slight blocking behavior when executed inside the primary server thread. This issue was resolved by moving the performance monitoring logic into an independent daemon thread, ensuring that client communication remained uninterrupted.

10. Conclusion

The GUI-based secure chat application was successfully enhanced for scalability, reliability, and maintainability. The introduction of **config.json** eliminated hardcoded parameters and simplified deployment across different environments.

Automatic client-side reconnection greatly improves user experience by masking temporary network failures, while graceful shutdown ensures proper socket closure and preservation of performance data.

Testing within the Mininet environment confirmed that the server efficiently supports 10 concurrent users, manages CPU and memory resources effectively, and maintains accurate session information even during unexpected network interruptions.

Overall, the implemented optimizations significantly improve the robustness and usability of the application while preserving the communication protocol developed in previous assignments.